

Optimizing Retrieval-Augmented Generation (RAG) for Geomechanical Engineering: Enhancing Specialized Knowledge and Domain Specific Language (DSL) generation capabilities for the Real-ESSI Simulator

Kopania, Grant Ramacciotti, Jiro
grant.kopania@gmail.com jiroramacciott@gmail.com

Wu, Albert
Albertwufamily@gmail.com

August 2025

Abstract

Large Language Models (LLMs) like ChatGPT have demonstrated excellent capabilities in natural language understanding and generation for general tasks. However, their accuracy declines when applied to specialized contexts, due in part to hallucinations and a lack of domain-specific grounding. This paper explores the integration of Retrieval-Augmented Generation (RAG) to improve the accuracy and relevance of LLM outputs in geotechnical engineering environments. We tested this with two different RAG applications. The first involves the development of a specialized retriever designed to index, embed, and retrieve data from lecture notes produced by Dr. Boris Jeremić. The second utilizes a Dynamic Few-Shot (DFS) prompting method to generate Domain Specific Language (DSL) code for the Real-ESSI Simulator (RES), a finite element analysis platform for soil-structure interaction. We used DFS to select the most relevant chunks of code to use as an example for the LLM to generate DSL code for the RES. We compare baseline LLM-generated code to code produced using the DFS. Using CodeBERT to measure semantic similarity and Longest Common Subsequence Similarity (LCSS), our results show that incorporating RAG yields higher alignment and accuracy when generating DSL code. Despite promising early findings, we note many limitations, including the short project duration and our inability to fully test the generated code with the RES. In parallel, the retriever showed improvement in retrieval accuracy and semantic precision in extensive, technical scientific literature through hybrid chunking enhanced with metadata and the novel Table of Contents (TOC)-based section stitching. However, it

still needs metric-based evaluation and performance improvement for further effectiveness and application. Nonetheless, this work highlights the potential of LLMs combined with RAG to bridge the gap between natural language interfaces and domain-specific engineering tools. Future work could further automate simulation workflows and expand this framework to other engineering domains requiring accurate and grounded reasoning.

DSL Generation source code is available [here](#).

Lecture notes retriever code is available [here](#).

1 Introduction

AI has quickly become a well-integrated and transformative aspect in numerous industries, including academia. Large language models (LLMs) have reshaped learning by making it personalized, streamlining feedback, and fostering engagement—making it a powerful educational tool and resource for students and teachers alike (Z. Li et al., 2025). AI has also proven itself a powerful catalyst for the discovery of new knowledge, demonstrating particular robustness in data-driven fields. An illustrative example of AI’s prowess is Demis Hassabis and John M. Jumper’s predictive algorithm, AlphaFold2, which can accurately predict protein structure based on amino acid sequence (B. Li & Gilbert, 2024).

Similarly, the specialized field of geomechanics within engineering is extremely data-driven, relying on intensive numerical models, technical documentation, and simulations (Fan et al., 2025). This field is particularly well-suited for AI-use, where natural language prompting can revolutionize access to information and simplify complex computational processes. Additionally, AI has significantly expanded the bandwidth of engineers by allowing them to assign software development tasks to increasingly capable LLMs (Hou et al., 2024).

The sweeping success and widespread application of LLMs can be explained by the efficiency of natural language prompting—allowing users to complete complex computational tasks by simply describing their tasks and goals. This method of communication through natural language is one of the most effective and intuitive ways for humans to interact with computers (Bekele, 2025). Our research builds on this principle, aiming to expand the capabilities of natural language prompting into highly specialized engineering tasks, particularly within complex simulation environments, which require specialized programming languages.

Despite these promising demonstrations, there remains a distinct gap in the application of LLMs for highly specialized engineering disciplines. This is especially true for the utilization of advanced simulation tools and domain-specific languages (DSLs). Fields like structural and geotechnical engineering demand a high degree of precision and expertise, and access to computational tools is often gatekept by specialized knowledge and large learning curves. The ability to utilize the Real-ESSI (Earthquakes, and/or Soils, and/or Structures and their Interaction) Simulator (RES), developed by Dr. Boris Jeremić, illustrates this challenge. The RES simulates realistic earthquake and seismic-soil-structure

interactions, providing valuable educational and research opportunities for students and researchers (Jeremić & Yang, 2024). It uses state-of-the-art nonlinear finite element analysis to model these geomechanical interactions and, unlike more widely-used engineering tools with graphical user interfaces (GUIs), the RES operates strictly through a DSL specifically for ESSI simulations. Because of this, users must have in-depth technical knowledge both of civil engineering principles and the program-specific DSL to take advantage of the RES. This creates significant accessibility barriers to a larger academic audience.

This presents a unique opportunity for progressing AI systems. Recent research, such as the development of GeoSim.AI has shown promise in this area, laying the groundwork for how natural language prompting can be utilized to bridge the gap between user intent through natural language to workable code for various simulators, written in popular languages like Python and Javascript (Bekele, 2025). Our work plans to build off of this concept, extending it to DSL generation. While DSL generation is not strictly new, our work aims to 1) increase its accuracy by eliminating syntax errors and reducing hallucinations, making LLM DSL generation reliable and 2) serve as a proof-of-concept by demonstrating that LLMs can support widespread generation of specific DSL code through various techniques.

While the GeoSim.AI platform serves as a versatile tool for code generation and scripting techniques, it lacks the capabilities of a typical Q&A-type chatbot. Additionally, its coding capabilities are more general and are not highly specialized. In addition to expanding code generation to DSL, our project aims to create a specialized chatbot, adapted to answer civil and environmental engineering questions, creating a wider and more encompassing portfolio of tools for engineers. This chatbot is specifically designed to use relevant engineering academic literature that include information regarding the usage of DSL code for the RES. As stated before, the accessibility of the RES is largely limited by access to engineering and DSL knowledge, and this chatbot aims to specialize in this domain-specific knowledge and address the lack of accessibility to the RES.

To accomplish this, our research investigates numerous approaches which aim to improve accessibility to specialized engineering tools using AI. The first approach will use a technique called retrieval-augmented generation (RAG). This works by providing additional reference information to LLMs to help build context. This enables LLMs to construct better tailored and informed responses to reduce hallucinations and increase specialization—something well-suited for a domain-specific chatbot. The second will be to train a model for (DSL) code generation from natural language enabling models to produce RES simulations based descriptive prompts, which will also use RAG. The generated DSL code will be evaluated through comparison with the base source-code, using qualitative and quantitative metrics. These metrics will measure the syntactic and semantic qualities of the code. In addition, the RAG-assisted DSL generation will also be compared to DSL code generated only with the description/prompt, which will act as a control.

The goal of these methods is to adapt AI’s capabilities to make specialized engineering tools more accessible, while also serving as a proof of concept for

the broader adaptability of LLMs in other technical, low-data domains. This paper aims to reinforce natural language prompting as an intuitive and powerful interface for tool interaction and knowledge accessibility, as exemplified in the GeoSim.AI project (Bekele, 2025). While GeoSim.AI demonstrated the viability of natural language for generation for general code, our work will focus on DSL generation—specifically the RES—and demonstrate how LLMs have the potential to reliably answer domain specific queries. This not only improves usability of tools like the RES, but also lays the groundwork for scaling this methodology across other specialized and technical fields.

2 Background & Related Works

The center of modern AI use comes in the form of generative pre-trained transformers (GPTs), like OpenAI’s ChatGPT and Google’s Gemini. GPTs function by generating responses based on diverse datasets built for general knowledge-based queries. While these models excel when utilized for general and conversational use, their accuracy and performance falls short when posed with domain-specific contexts and questions (Lewis et al., 2020). Its tendency to hallucinate—generate plausible but incorrect information—is simply a more detrimental consequence of its lack of specialization, occurring primarily when answering domain-specific questions. This presents a significant deterrent for the use of LLMs and GPTs specifically in education and assistance in fields like civil and structural engineering, where accuracy is crucial.

GPTs differ from other LLMs in their organization and consequently their functionality. Compared to other LLMs, GPTs are creative and open-ended, comparatively proficient in creative expression, code generation, and idea generation (Sufi, 2024; Yenduri et al., 2023). This is largely due to their decoder-only architecture; when generating a response, GPTs are designed to predict the next most likely token—a small bit of textual information—making GPTs reliant on probability (Yenduri et al., 2023). Bidirectional Encoder Representations from Transformers (BERTs) are another popular type of LLMs whose structure largely eliminates the possibility of hallucinations. BERTs, however, do not have the creative capabilities that GPTs do. They are accurate and are unlikely to hallucinate, but can only comprehend text, and not recreate free-form text like stories or code. Their bidirectional process of token creation prevents the plausible yet incorrect nature of hallucinations; BERTs use masked language modeling, predicting the next tokens by using context to the left and right (i.e. bidirectional). Hybrid approaches and techniques have been explored, such as RAG, which mitigates hallucinations while preserving GPTs’ generative capabilities, and showing great promise, especially when applied to domains where accuracy is crucial (Bécharde & Ayala, 2024; Lin et al., 2022a).

Hallucinations are an extremely major concern, especially in fields such as medicine and law, where the use of specialized chatbots are prevalent and have detrimental effects on human lives and costly monetary consequences (Charlotin, n.d.; Yenduri et al., 2023). Some legal chatbots have been found to hallucinate

from 58% to 88%, depending on the model (Dahl et al., 2024). RAG stands to help LLMs combat this extremely high hallucination rate by providing a framework which accesses external documents or domain-specific contexts to ground its response rather than relying on its general pre-trained data from which it probabilistically generates its response. GPTs prioritize fluency and likelihood rather than accuracy; RAG brings additional context that informs the generation, preserving the creative liberties that GPTs take while remaining accurate and ideally increasing specialization in domain-specific queries (Lin et al., 2022b).

DSLs are pre-trained on large sets of data, posing a significant challenge of code generation for low-resource programming languages (LRPLs) and DSLs, due to the underrepresentation in training data. This has led to a consequent drop in performance and accuracy, compared to high-resource programming languages (HRPLs), such as Python and Java, which have extensive documentation, community support, and an excess of data (Cassano et al., 2024). This trend holds true over various LLMs and code generation models, which consistently perform more accurately with HRPLs in comparison to LRPLs (Joel et al., 2024). Furthermore, DSLs are specifically data-scarce and have received limited LLM research focus, due partly to an overall disparity in available datasets and established benchmarks (Orlanski et al., 2023; Pujar et al., 2023). This lack of standardized material has slowed the advancement and progress of code generation for LRPL and DSL (Zhang et al., 2024). Additionally, inherent uniqueness of DSLs—each being made and used for their own specific purposes—adds further complexity to their generation. DSL syntax and semantics, the equivalent of grammar and meaning, varies greatly case-by-case, making DSL generation a far greater challenge than HRPL code generation. The great variety of DSLs make it difficult to generalize code generation processes, complicating the development and scalability of code generation models compared to those used for HRPLs.

3 Methodology

In this section, we will provide an overview of the different model architectures we use, and in the following sections we will elaborate on the details of each of these approaches.

3.1 Applying RAG for Knowledge Retrieval: An Experience Report

3.1.1 RAG Framework & Application

The naive RAG framework shares a common process, and according to the works of Khan et al., this process goes through six major steps before a final output is returned. (Khan et al., 2024). First, raw data is collected in the form of online documents such as PDFs or LaTeX files, among others. Then, that raw data is

pre-processed through the initial stage of the RAG framework. It is here that the data is cleaned and structured into small units, or chunks, which can be indexed and retrieved later down the RAG pipeline. After, the tokens/chunks are converted into vector representations using embedding models and are stored in a vector store; these vectors hold semantic meaning related to the text. Then, the retriever of the framework converts the user’s query into a vector embedding, searches for similarity between the query vector and token vectors, and pulls the most semantically relevant chunks for the LLM to utilize. Combining this domain-specific knowledge with the fixed-knowledge of the LLM, the framework is then able to effectively augment the context behind the query. Lastly, the relevant context, tokens, and query are given to the LLM which generates an improved, factually grounded response.

It is this process that lays the groundwork for our application of a RAG-enhanced AI designed to pull data from extensive, detailed scientific literature. Using this initial framework, we aimed to further improve and specialize the retriever from the RAG system to accurately pull relevant information from Dr. Jeremić’s 3287 page lecture notes. RAG within the civil and environmental engineering context provides domain-specific, precise results which are especially critical when interpreting computational mechanics or engineering models. Considering that this application requires the retrieval of chunks from such an extensive and technical piece of scientific literature, the RAG bot was specifically tailored with this in mind.

The goal of this section is to provide accessible, concise, and thorough details of the development of this RAG portion of the AI assistant for future validation, reproduction, or continuation of this study. This includes the design philosophy, retriever behavior, file structure, design changes, future advice, and final design. Please note that due the exploratory nature of this research along with limited oversight, much of the design philosophy behind this first methodological stage was trial and error. We mainly focused on improving the accuracy and semantic similarity of chunks pulled by the retriever to the user query. This methodology reflects a repetitious approach focusing on performance.

3.1.2 RAG Design Process

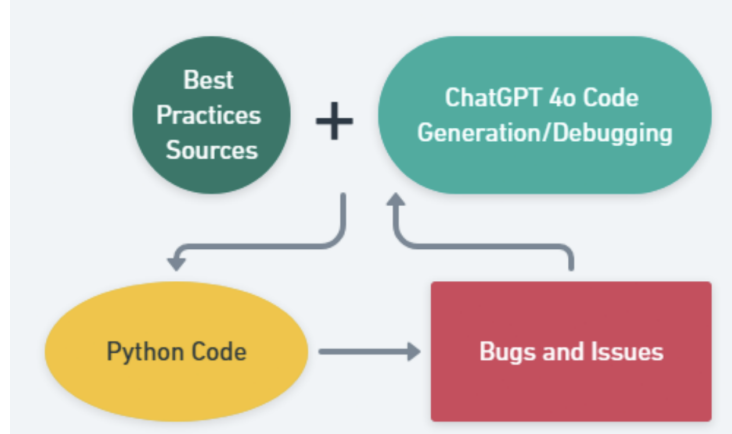


Figure 1: RAG Design Process

The design process behind the retriever is shown in Figure 1, and is as follows. Using PyCharm as a medium for coding and bugfixing, the ChatGPT 4o model was utilized interactively to aid in debugging and design exploration. The model was used to produce code relevant to the problem. Before the chunk-query accuracy testing phase, ChatGPT 4o provided recommendations and coding suggestions in order to bug-fix errors relating to file formatting or inconsistencies with the lecture notes PDF format. When in the accuracy testing phase, knowledge from ChatGPT 4o and best practices sources such as the works of Khan et al. and Wang et al. (2024) were used to implement improvements to the retrieval code to increase chunk-query semantic similarity. This progression was implemented to conduct multiple improvements over the course of the study rather than integrating improvements linearly.

3.1.3 Retriever Behavior

In order to specialize the retriever to the lecture notes, we decided on hardcoding the embedding and indexing system instead of using open-source systems such as LlamaIndex or LangChain. Even though these open-source systems have seen to be effective for prototyping RAG systems, we found hardcoding the chunking and retrieval system necessary for its specialization towards extensive, technical literature. This allowed us increased granular control over metadata and chunking logic needed to accurately segment the dense structure and complex formatting that open-source software would usually abstract.

We used the all-miniLM-L6-v2 SentenceTransformer model for embedding due to its balance of speed, semantic accuracy, and low cost. Embeddings were stored in a custom vector store and serialized with pickle. This allowed us greater flexibility and storage modification needed for the specialization of

the system. The manual control over embedding, indexing, and retrieval also enabled custom metadata filtering as well as full debugging logic necessary for testing and improvement.

The chunking logic of the system evolved over time as more format issues presented themselves; however, the foundation of the logic was hybrid chunking with parsing from the PyMuPDF Python library. The hybrid chunking style used in our RAG system combines both structural and semantic chunking. We chose hybrid chunking as it allows for semantic coherence in combination with the structure of the PDF, rather than semantic chunking or structural chunking individually which may produce chunks without full context or semantic similarity respectively.

Structural Chunking relies on the structure within a document or database (e.g. section headings, pages, paragraphs, sentences), to determine chunk size and boundaries. This style is helpful due to the hierarchical structure of the document, but may cut off semantically similar elements.

Semantic Chunking relies on the semantic similarities between groups of text to determine chunking. This aids in preserving local meaning, but may cut off the larger context behind groups of text and mathematical equations.

Hybrid Chunking combines the structural components with the semantic connection between groups of text. This uses the structure of the document as the foundation for chunking, but applies semantic-aware splitting for maintaining context. This hybrid approach was achieved by utilizing embeddings affected by both metadata, which describes section, section number, page, source, etc. of the chunk, and parsed pages and paragraphs. Other additions to enhance this hybrid approach included the use of a Table of Contents (TOC) parser for section mapping and chunk stitching as well as fallback UTF-8 encoding to recover special characters.

This logic first started with page-level chunking, however sentence/window overlap quickly followed as a process to achieve increased semantic precision. Due to the hierarchical and lengthy nature of the sections within the lecture notes, an ultra-hybrid style was introduced to capture sentence level, paragraph level, and page level chunks. This, however, was soon removed for section and page level segmentation. This was due to the formatting of the document and the numerous multi-page sections. Finally, we introduced section-stitching with a TOC parser to stitch chunks within the same section together.

Cosine similarity scores were used to quantify the semantic similarities between the chunk vectors and the query vector. These scores were used to determine what chunks were outputted based on how high their cosine similarity scores were.

We initially tested with small, local LLMs such as the facebook/opt-1.3b model. Their small token sizes led us to try OpenAI’s ChatGPT 4o and 3.5 turbo API models, however due to monetary constraints, we decided to focus on improving the accuracy of the output chunks purely from the retriever. This allowed us to evaluate the accuracy of chunks independently of LLM generation.

To the best of our knowledge, the TOC-based section stitching applied to this retriever seems to be a novel approach in structural chunking. While other

works have described developments in TOC recognition for parsing (Wu et al., 2013), advanced contextual chunking strategies (Merola & Singh, 2025), and hierarchical chunking strategies (Nguyen et al., 2025), we have not found a RAG framework in academic literature that uses its own table of contents for structuring chunk-stitching of multi-page sections into singular units. The closest source we have found is of the work of Kang et al., which uses a system that retrieves or generates a TOC for provided documents to use for chunk boundaries (Kang et al., 2024). Differing from this top-down approach of dividing text into chunks that fit structured sections of a document, TOC-based section stitching applies a bottom-up approach. After text is split into chunks, it is then stitched together to form semantically coherent units based on true section boundaries. This may be less efficient than the top-down approach, however this style of chunking may improve retrieval precision by mapping one-to-one semantic units without accidental context cut off.

3.1.4 Extra Developments

During our experimentation of different retrieval methods, we implemented several improvements to enhance chunk similarity and specialize the retriever towards the specific details of Dr. Jeremić’s lecture notes.

- Custom Output Behavior enabled flexible analysis and comparison of chunk outputs. Users could either 1: view all output chunks in the Py-Charm console, or 2: download them to a “retrieved_context.txt” file in the user’s downloads folder for review or integration into LLM queries.
- Cosine Similarity Score Boosting and Penalizing while pulling the top_k allowed for chunk similarity adjustments. Using keywords, regex code, or certain metadata, certain chunks that included author emails, references, or specific sections were nullified (similarity *= 0), penalized (similarity *= 0.4), or boosted (similarity *= 1.4) respectively.
- Regex-Based Filtering for email pattern, affiliations, and section headers was also implemented during preprocessing to remove non-informative content in case text bypassed scoring penalties or nullification.
- Equation Extracting preserved LaTeX-style and block equations to maintain mathematical content.
- Style Heading Recognition aided structural chunking quality by identifying font size differences and weight.
- Caption Preservation kept full figure and table captions for increased semantic context.

3.1.5 Lessons Learned

During the development of the retriever system, we gained many insights that enhanced our approach and increased our understanding of PDF-related indexing, chunking, and retrieval. These insights emerged from trial and error process and iterative adjustments to our source code.

A key insight was adapting structural chunking logic correctly to the size and structure of the document. We initially tested smaller segment sizes such as sentence and paragraph chunking or sentence/window chunking for increased semantic precision. However, these caused overly fragmented output chunks that lacked whole context, so we experimented with a combined section-page level chunking approach. This eventually led to the use of TOC-based section stitching which greatly aligned concept context with the structure of the document. Due to the many multi-page sections within the lecture notes, we found that chunking logic scaled to an extensive amount of text was most effective for retrieval accuracy. A best practice would be to adjust the chunking logic to the size, complexity, and format of the document being indexed.

Initial efforts were focused on improving performance of the retriever before implementing the lecture notes. Many adjustments were made to increase retrieval accuracy and semantic precision based off of an example scientific paper of 15 pages. This led to a lack of efficiency in specializing the retriever towards the more extensive document encasing multiple PDFs rather than a one PDF document. While an efficient approach is to use an example document to prototype the RAG system, especially a less substantial document, more effort could be spent specializing the system to the document. A best practice would be to implement the more extensive document quickly. This approach allows for more effective development specializing the system to the finer formatting of the document.

Lastly, the magnitude of effort put in towards parsing the TOC through an automatic parsing script was great. Hidden text, headers, footers, and page numbers within the format of the PDF made parsing the TOC taxing, not to mention the quantity of entries needed to be parsed. A best practice to include in TOC parsing would be to utilize the .toc file generated when converting the LaTeX file of the document to a PDF, or if possible, remove extra unnecessary text from the table of contents pages altogether.

“Retrieval accuracy” and “semantic precision” were assessed manually through comparing the section and pages of the retrieved chunks to those described by the user query. For information and examples of evaluation, look in the project documentation¹.

RAG Project Documentation

3.2 Dynamic Few Shot Prompting via RAG

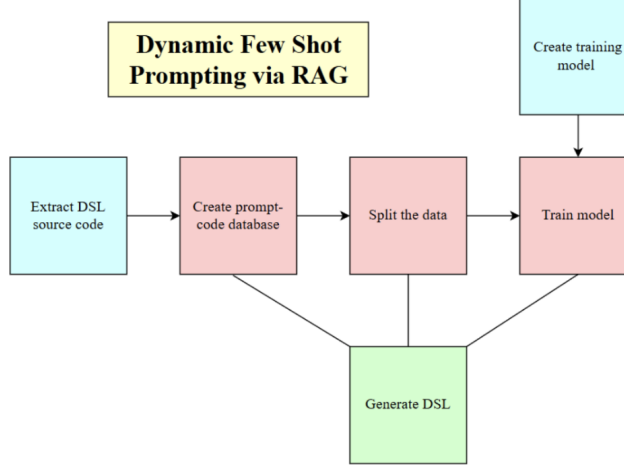


Figure 2: Dynamic Few Shot Prompting Workflow

Our approach to generate RES-specific DSL code consisted of two components: 1) the prompt-code pair database of RES examples and 2) the model itself, capable of dynamically retrieving and formatting the examples (within the dataset) for few-shot inference and generation. The central objective was to train the system that could generate syntactically valid Real-ESSI DSL when given NL prompts describing the simulations.

To generate usable and accurate DSL code for the RES, we designed a dynamic few-shot prompting model built using a RAG framework. We then constructed the database of prompt-code pairs. These were then split into distinct datasets: one set used for the model’s few-shot reference to train on and another test set to evaluate the generated DSL outputs (see Figure 2). The generated DSL was then evaluated against the original source-code using a combination of similarity metrics, including using vector similarity and Longest Common Subsequence Similarity (LCSS) match, to qualitatively and quantitatively assess output fidelity without actually using the generated code within RES (attempting to assess the code using RES was not feasible due to the scope of this research).

3.2.1 Prompt-Code Dataset Construction

In order to construct an effective RAG system for RES code generation, we first had to construct a dataset of examples of prompt-code pairs for our system to retrieve from. The dataset was based off of a collection of source codes created by Professor Jeremić’s for the RES. The initial collection consisted of

151 simulation projects and a total of 435 .fei source files. Each mini-project directory contained one or more simulation files written in the Finite Element Input (FEI) language.

The final database contained pairs of NL prompts/descriptions a user might give the model in order to generate the paired pre-existing source code provided by Dr. Jeremić. These serve as examples to help guide and train the bot (will be explained in the next section). To generate natural language prompts describing each project, we used a script that loaded all readable files into a project folder. If the total combined length of all files was under a 10,000-character limit, we used all of the files. Otherwise, the script extracted a single representative file—typically the main file identified by file-name heuristics—to ensure the prompt remained within token limits, while retaining the most essential information.

Each NL prompt was constructed using a consistent template that instructed the GPT-4 model gpt-4-0613 to generate a concise description that a user might give to a language model to recreate the associated simulation code:

Task: Based on the provided file(s), generate a concise and specific prompt that someone would use to ask a language model to produce these files from scratch. Provide only the prompt text.

+ [DSL source code]

The prompt generation process used the OpenAI API, and the outputs—NL prompts and corresponding DSL code—were stored in a JSON file.

The final dataset consisted of roughly 150 prompt-code pairs. We applied a standard 80/20 train-test split, keeping 120 examples for “training” in the retrieval database and 25 examples as a “test” evaluation set. This split is conventionally used for machine learning and allowed us to adequately evaluate our model’s performance on unseen code-generation tasks while maintaining a sizable and realistic retrieval collection.

To further improve our system’s effectiveness, we explored two extensions to the dataset construction pipeline. First, we first considered implementing leave-one-out cross-validation (LOOCV), a method allowing each data point to serve once as the test set while the remaining 150 pairs were used for training and retrieval. This would have provided a more robust evaluation and increased the raw number of examples available (due to the relatively small amount of pre-existing data) for retrieval during reference, compared to the traditional 80/20 split. But, this would have taken a considerably larger amount of time, given that we would have to iterate through each of the different test sets, so we decided against it. Second, we aimed to improve the quality of the NL prompts by refining the GPT-4 generation prompt. We provided a more specific prompt and more detailed instructions to elicit more specific and informative descriptions:

Task: Based on the provided code file(s), generate a highly detailed and specific natural language prompt that a developer could use to accurately reproduce the same functionality using a language model.

+ [DSL source code]

The prompt should:

- Clearly describe the purpose and functionality of the program
- Specify any libraries, packages, or frameworks used
- Describe the inputs, outputs, and processing logic in detail
- Mention any assumptions, constraints, or configurations
- Reflect the structure of the code (e.g., number of files, key functions, classes, or modules)
- Use clear, natural language phrasing as if explaining to a competent developer or AI model
- Do not include the code itself. Output only the resulting prompt text. Do not use any special characters

We wanted the descriptions to be as detailed as possible in order to retain as much accuracy throughout the transcription process. These improvements were expected to enhance both the contextual relevance of retrieved examples and the overall accuracy of generated code. This did take up a lot more time than the original generated prompts, but we believed that an increase in specificity would be worth the time.

3.2.2 Dynamic Few Shot DSL Generation

Our methodology is built on dynamic few-shot prompting, an expansion of few-shot prompting mechanisms. In standard few-shot prompting, a set of exemplars are embedded into the prompt to guide and condition the language model’s output. Few-shot prompting has proven to help LLMs for specialized downstream tasks, especially when task-specific data is limited in the pre-training dataset (Brown et al., 2020; Gao et al., 2021). However, its effectiveness is constrained by the context window size: for instance, GPT-4-Turbo-0125 has a context token limit of 128,000 tokens, which effectively caps the number and complexity of examples that can be used. To work around this limitation, we implemented a RAG-based retrieval pipeline that dynamically selects and formats the most relevant few-shot examples from our prompt-code database for each input prompt. This ensures that the language model receives maximally informative and domain-relevant guidance for each individual generation task.

This strategy has been previously applied to generic DSL generation tasks, such as UML sequence diagrams (Ferrari et al., 2024), but to our knowledge, we

are the first to apply it to the RES. Unlike general-purpose programming, where logic and syntax complexity are major challenges, the primary difficulty in RES DSL lies in encoding engineering domain knowledge: understanding how physical parameters, material models, and boundary conditions map into simulator-specific syntax. Our dynamic few-shot prompting approach was specifically designed to bridge this gap, enabling the LLM to incorporate example-driven domain knowledge into its output generation, ultimately producing executable and semantically correct Real-ESSI code.

We first mounted and loaded the dataset in Google Colab. Each item in the dataset contained a project identifier, an NL prompt, and its matching RES source-code. Using the sentence transformer model all-MiniLM-L6-v2, we generated embeddings for both the training and test subsets that captured its semantic content. These embeddings were used in a high-dimensional vector-space. Cosine similarity was used to compute test prompts NL semantic closeness to training prompts. Cosine similarity measures the angle between two vectors in a multi-dimensional space, enabling the system to reliably select the most contextually relevant example to use as reference, rather than simply basing similarity based off of absolute distance within the vector-space. The training example with the highest similarity was selected as the best match to be used for reference. The NL prompt and DSL code were then extracted and formatted as an in-context example.

To maintain semantic guidance without exceeding token limits, the selected RES code was truncated when necessary. The final input was constructed as a template:

Instruction: Here is a relevant example of a prompt and good Real-ESSI code

+ [retrieved training prompt]
 + [retrieved training DSL code]

Learning from that task, can you implement code for this?
 + [test prompt]

This prompt structure kept the task clear and the code examples relevant, helping the model understand both what is being asked and providing a similar and relevant example to create the DSL. By combining embedding-based retrieval with example-embedded prompting, our system aimed to provide GPT-4 with examples tailored to each DSL generation query, with the goal of improving the accuracy and relevance of the generated RES code.

3.2.3 Evaluation Methods

To evaluate the performance of our system pipeline, we first adopted a source code similarity analysis approach. This method assessed how closely the generated code matched the original source code. Since each test set sample included

a known ground truth—i.e. The original source code —this allowed us to directly compare the actual vs expected outputs.

We used CodeBERT, a pre-trained transformer model built on the BERT architecture and specialized for programming languages, to embed both the generated and ground truth code into vectors. CodeBERT was trained on a large database consisting of NL-code pairs, helping the model learn both the syntax and semantics of code across different programming languages. The model is capable of understanding token-level dependencies and higher-level code semantics, making it well-suited to analyze our code qualitatively without human analysis.

To implement, we tokenized both the generated code and the ground truth code which were then used to create embeddings for each piece of code. These embeddings captured the structural and semantic features of the code (like function usage, variable content, etc.). We then computed the cosine similarity between the two embedding vectors. This similarity score serves as a baseline qualitative score for our code.

In our second method of evaluation, we applied a quantitative-based comparison using the Longest Common Subsequence Similarity (LCSS) method. This method is based on syntactic structure rather than semantics of the CodeBERT evaluation. Each piece of code, both ground truth and generated, was tokenized using a special tokenizer that parsed the DSL by separating identifiers, symbols, keywords, etc. The tokenized sequences were then passed into the LCSS algorithm, which computed the longest matching subsequence of tokens between the base and predicted outputs.

The normalized LCSS score was then calculated by dividing the length of the common subsequence by the maximum length of either code sequence. This captures overlap even with reordered or added tokens, which other conventional string-based evaluation metrics may penalize. This method provides a concrete measure of syntactic accuracy, with higher scores reflecting token-level similarity and structural preservation, conveying the degree of the accuracy of the overall structure of the code.

We compared the RAG-assisted DSL generated code against the original source code using the following metrics in addition to comparing the RAG-assisted code against a control. The control was DSL code generated solely by inserting the description/prompt (which contains a request to construct DSL code based on the description).

4 Results

4.1 Semantic Similarity Using CodeBERT

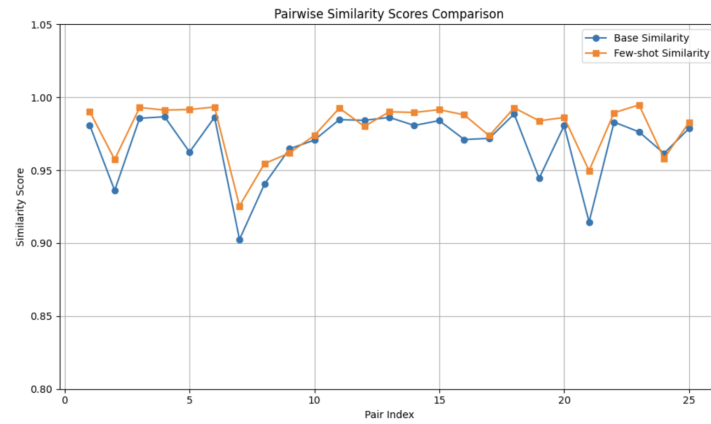


Figure 3: CodeBERT Semantic Similarity Scores Visualized.

Note. Both baseline ChatGPT code and Few-Shot were compared to the original source code.

Pair	Base ChatGPT	Few Shot
1	0.9810	0.9903
2	0.9362	0.9574
3	0.9857	0.9930
4	0.9867	0.9912
5	0.9625	0.9917
6	0.9863	0.9933
7	0.9026	0.9255
8	0.9404	0.9546
9	0.9647	0.9618
10	0.9707	0.9739
11	0.9847	0.9926
12	0.9842	0.9801
13	0.9862	0.9901
14	0.9807	0.9897
15	0.9840	0.9915
16	0.9711	0.9880
17	0.9719	0.9737
18	0.9886	0.9928
19	0.9445	0.9839
20	0.9806	0.9862
21	0.9146	0.9497
22	0.9830	0.9894
23	0.9763	0.9948
24	0.9615	0.9582
25	0.9786	0.9827
Avg. Similarity	0.9683	0.9790
Min. Similarity	0.9026	0.9255
Max. Similarity	0.9886	0.9948

Table 1: CodeBERT Similarity Scores: Base ChatGPT vs. Few Shot.
Note. Base ChatGPT is the control. Few Shot represents the code generation through the RAG pipeline

To assess the semantic accuracy of the generated code (both the baseline and RAG-assisted), we employed CodeBERT to calculate pairwise similarity scores. We compared two approaches: the baseline ChatGPT model prompted without additional guidance (only the prompt, acting as the control), and a few-shot RAG-assisted version that included relevant examples of RES code retrieved from the test database.

Across the 25 evaluation sets, the few-shot RAG-assisted model consistently produced code with higher semantic similarity. The average similarity score for the baseline ChatGPT output was 0.9683, while the few-shot approach achieved an average of 0.979, making an approximate 1.1% absolute improvement. Roughly 3 sets performed about the same or worse, but the majority of the evaluation set performed markedly better (see Figure 4). This suggests that

a more specific and generally more coherent prompt can lead to more consistent and cohesive outputs, as measured by semantic similarity (however small the increase may be). The semantics, or meaning behind the code, remained relatively the same, which was to be expected. The key struggle is not the structure or the semantics behind the code, but rather the syntactic details. LLMs like ChatGPT are excellent at processing and understanding language, but struggle with fine details often (Sun et al., 2024).

In addition, the minimum similarity score improved from 0.9026 (the baseline; control) to 0.9255 (few-shot), and the maximum increased from 0.9886 to 0.9948, indicating a stronger consistency and a higher ceiling for semantic preservation using the dynamic few shot RAG prompting. All 25 scores are provided in Table 1.

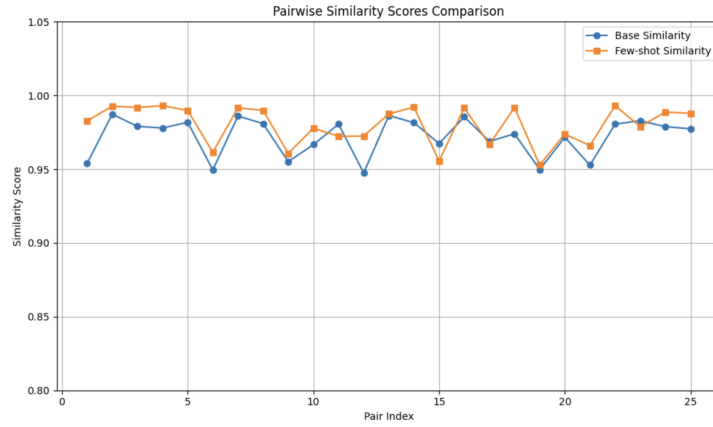


Figure 4: CodeBERT Semantic Similarity Scores Visualized (using Detailed Prompts)

Note. Both baseline ChatGPT code and Few-Shot were compared to the original source code.

Pair	Base ChatGPT	Few Shot
1	0.9540	0.9827
2	0.9873	0.9928
3	0.9791	0.9919
4	0.9779	0.9931
5	0.9818	0.9899
6	0.9496	0.9615
7	0.9861	0.9916
8	0.9809	0.9899
9	0.9551	0.9607
10	0.9667	0.9779
11	0.9805	0.9723
12	0.9475	0.9725
13	0.9864	0.9875
14	0.9816	0.9921
15	0.9674	0.9555
16	0.9855	0.9917
17	0.9689	0.9668
18	0.9740	0.9919
19	0.9497	0.9528
20	0.9716	0.9739
21	0.9528	0.9660
22	0.9806	0.9931
23	0.9829	0.9789
24	0.9788	0.9888
25	0.9774	0.9879
Avg. Similarity	0.9722	0.9801
Min. Similarity	0.9475	0.9528
Max. Similarity	0.9873	0.9931

Table 2: CodeBERT Semantic Similarity Scores (using Improved Prompts)

Using the improved prompts (mentioned in section 4.2.1), we reapplied the same metric and tests. Across the 25 test pairs, the few-shot RAG-assisted model consistently scored with higher semantic similarity scores than the baseline code. The average similarity increased from 0.9722 for the baseline to 0.9801 for the few-shot approach, showing an approximately 1% increase (0.8%) in semantics. The average similarities had an overall increase, which was expected, but led to a smaller difference between the baseline and RAG-assisted code. The improved prompt benefited the baseline, signifying that the RAG-supplied example provided good enough reference for generating the code. In addition, this implies that benefits of using a detailed prompt are capped, especially for semantics, where both models achieve extremely high matches. The minimum similarity score also increased from 0.9475 to 0.9528, with a roughly 1% increase. The maximum similarity score increased from 0.9873 to 0.9931, closing in on a perfect semantic match (see Table 2). Similar to the original prompts,

the few-shot generated code performed consistently better than the baseline, with only a few points performing worse (see Figure 5).

4.2 Evaluation Using Longest Common Subsequence Similarity (LCSS)

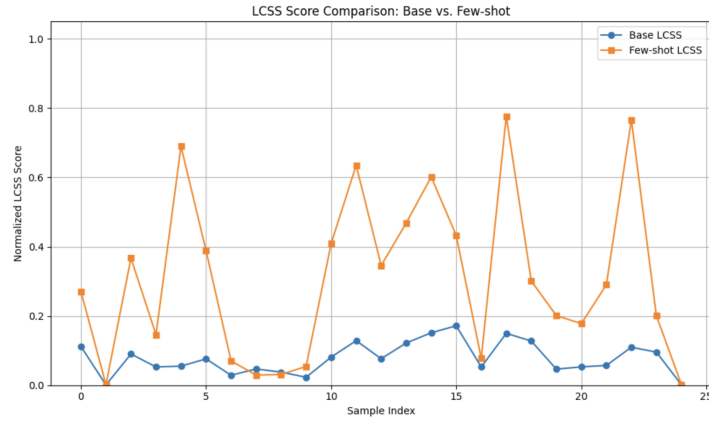


Figure 5: LCSS Scores Visualized

Pair	Base ChatGPT	Few Shot
1	0.1120	0.2707
2	0.0009	0.0018
3	0.0901	0.3669
4	0.0529	0.1461
5	0.0552	0.6898
6	0.0762	0.3888
7	0.0288	0.0704
8	0.0470	0.0292
9	0.0378	0.0311
10	0.0231	0.0544
11	0.0812	0.4090
12	0.1287	0.6350
13	0.0765	0.3455
14	0.1220	0.4682
15	0.1514	0.6014
16	0.1716	0.4323
17	0.0528	0.0775
18	0.1499	0.7768
19	0.1277	0.3009
20	0.0465	0.2011
21	0.0530	0.1779
22	0.0572	0.2901
23	0.1097	0.7659
24	0.0957	0.2005
25	0.0014	0.0022
Avg. Similarity	0.0780	0.3093
Min. Similarity	0.0009	0.0018
Max. Similarity	0.1716	0.7768

Table 3: Normalized LCSS Scores (Original Prompt)

The normalized LCSS metric showed notable differences when using the few-shot prompting compared to the baseline. The average similarity increased from 0.0780 to 0.3093, nearly a four-fold increase indicating the preservation of ordered subsequences, signifying a large increase in the retention of accurate syntax. This implies that the selected examples provided a good guideline for the syntax. The maximum similarity score also rose substantially, from 0.1716 to 0.7768. The minimum similarity also rose slightly from 0.0009 to 0.0018, showing improvement even in the weakest-performing test cases (see Table 3).

Several pairs demonstrated dramatic gains in performance, such as Pair 18 and Pair 23 which increased from 0.0528 to 0.0775 from 0.1097 to 0.7659, respectively. Only a few performed worse and by a small margin, like Pair 8 and Pair 9. The few-shot generated code performed better throughout, with many instances of improvement by a significant margin (see Figure 6).

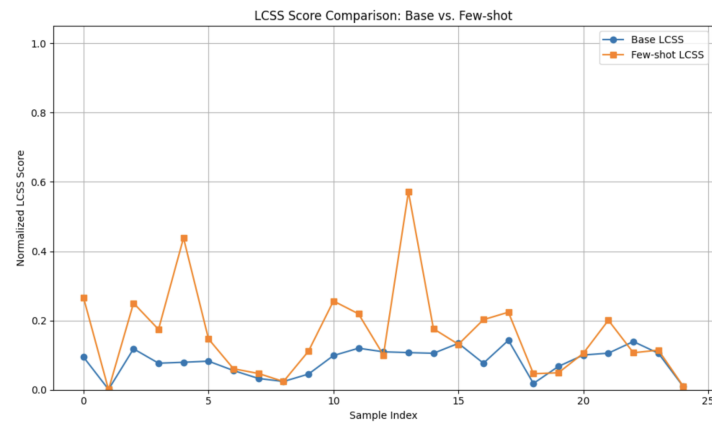


Figure 6: LCSS Scores Normalized and Visualized (using Detailed Prompts)

Pair	Base ChatGPT	Few Shot
1	0.0943	0.2657
2	0.0010	0.0005
3	0.1182	0.2503
4	0.0767	0.1743
5	0.0795	0.4383
6	0.0826	0.1468
7	0.0555	0.0607
8	0.0326	0.0470
9	0.0240	0.0239
10	0.0453	0.1114
11	0.0988	0.2563
12	0.1200	0.2194
13	0.1096	0.1001
14	0.1073	0.5714
15	0.1054	0.1762
16	0.1340	0.1312
17	0.0766	0.2024
18	0.1429	0.2239
19	0.0182	0.0464
20	0.0674	0.0491
21	0.1003	0.1056
22	0.1057	0.2008
23	0.1392	0.1069
24	0.1054	0.1146
25	0.0088	0.0098
Avg. Similarity	0.0820	0.1613
Min. Similarity	0.0010	0.0005
Max. Similarity	0.1429	0.5714

Table 4: Normalized LCSS Scores (using Detailed Prompts)

Using LCSS, the improved prompt setup resulted in relatively modest gains over base ChatGPT responses compared to the original prompting. The average similarity improved from 0.0820 to 0.1613 between the two models (baseline and fewshot respectively). This represents nearly a 98% increase, a much smaller degree than in the original prompt comparison (which nearly a 300% increase). The maximum similarity for the improved prompt set was 0.5714 (Pair 14), which was significantly higher than the base model’s max of 0.1429. Conversely, the minimum similarity values were 0.0010 and 0.0005. These relatively low similarity maximum and minimum represent a large disconnect between the original source code and generated, throughout both types of generations. Unlike the original prompt comparison, the improved prompt shows clustering and decreases between methods across many pairs—such as Pairs 8, 9, 13, 16, 20, 21, and 24. Pairs 5, 11, 14, and 18 still show clear increases under few-shot prompting.

5 Discussion

5.1 Dynamic Few Shot Discussion and Analysis

Dynamic few shot learning (DFSL) consistently improved code accuracy when compared to its baseline large language model (ChatGPT 4.1) for the Real-ESSI coding language, a DSL which the LLM had not encountered in the pre-training phase. This advantage persisted regardless of prompt quality, with the concisely prompted DFSL beating out the base LLM even with a significantly more detailed and specific prompt. DFSL only slightly increased the average of CodeBERT similarity, which is to be expected as LLMs are already capable of capturing high-level semantic intent even without additional prompting. Our main concern is the LCSS metric which is more indicative of a model’s capability to adhere to syntactic complexity. This is the key challenge for LLM’s on unencountered DSL’s, and DFSL significantly improves the average LCSS length from 249 to 637 for less specific prompts. The improvement from using a detailed prompt as opposed to a concise prompt is relatively insignificant, which suggests that relevant in-context examples play a far more critical role than prompt verbosity for syntax accuracy. We find that normalized LCSS scores actually decrease with detailed prompts, but this is because they produce longer outputs which reduces the normalized score despite absolute LCSS length slightly improving.

Model Configuration	Avg CBS	Avg LCSS Length	Avg Norm. LCSS
Concise Prompt – Base LLM	0.968	249.16	0.078
Concise Prompt – Few-Shot (FSL)	0.979	636.56	0.309
Detailed Prompt – Base LLM	0.972	271.88	0.082
Detailed Prompt – Few-Shot (FSL)	0.980	707.52	0.161

Table 5: Results Summarized

While the use of DFSL seems promising and netted an increase in accuracy, because we did not use a control of only few-shot code generation—supplying a randomly-selected example code along with the prompt—we do not know how effective the RAG-determined example actually was in the over DSFL pipeline. In addition, because of the limited token window on what could be supplied as a guideline in few shot, we could only select one example, and sometimes less (only the main part of the example). This, paired with a small database, can explain why some pairs performed significantly worse or better than others; some may have had a very similar example, while others did not. Being able to supply multiple relevant examples would have likely led to a more consistent and improved performance.

As previously mentioned, we considered using LOOCV (to split our database for training and testing), which would increase the number of training examples available to reference, given the relatively small size of the dataset. The traditional 80/20 dataset is conventionally used with larger datasets, something we

did not have the privilege to use, but was the most efficient way to split the data. The LOOCV usually leads to more thorough testing, but would require time-consuming iteration through the entire dataset to produce the results we achieved with the 80/20 split. Our time within our research was largely limited, restricting us from evaluating the generated code within the RES. While we successfully developed and refined prompts that showed promising improvements in output consistency and structure, we couldn’t test the functionality and accuracy of the generated code within the simulator. This means that whether the generated code—despite appearing accurate and promising—would run as intended or produce valid simulation results is still unknown. We could evaluate the semantics of the code with reasonable confidence, but we do not definitively know the accuracy of the code until it is used in RES (i.e. we are yet to know if the prompt matches the simulation). Furthermore, the best measurement of syntax accuracy would be from the RES itself; if the code is not syntactically correct, the RES would not run.

5.2 RAG for Specialized Chatbot: Future Research

The goal of the specialized retriever adapted to Dr. Jeremic’s lecture notes is to provide a basis for future improvements and research regarding accurate retrieval of key concepts and sections. While the system improved greatly over the course of the study with the implementation of hybrid chunking, TOC-based section stitching, and cosine similarity score modulation, there are several areas of study that could be researched further. These areas include new methods to increase accuracy, the application and testing of AI generation in combination with the retriever, and other best practices that were unused.

While this project mainly focuses on optimizing the accuracy and performance of the retriever pipeline, complete RAG systems include an AI model for answer generation. Integrating an external LLM via API as a natural extension to this work could enable further evaluation of end-to-end performance (e.g. GPT-4). Using this approach, the effectiveness of TOC-based section stitching could also be validated with metrics and data analysis such as BLEU for output similarity, hallucination analysis resources, or MRR for retrieval accuracy. Future research could also include utilizing prompt-engineering, cross-encoder reranking and pseudo-relevance feedback (H. Li et al., 2023; Mishra et al., 2024). Utilizing an embedding model such as SciBERT could also potentially be an avenue for improvement as it may outperform the embedding model used in the study. Through developing a complete end-to-end RAG system with advanced assessment metrics, future RAG-enhanced AI systems could better improve retrieval and response from highly technical scientific literature.

6 Conclusion

Our findings demonstrate that DFSL meaningfully enhances the performance of LLMs on DSLs they were not exposed to during pre-training. DFSL significantly

improves syntactic fidelity, as reflected in the LCSS metric, particularly under concise prompting conditions. Notably, prompt specificity had limited impact on syntactic accuracy, reinforcing the importance of relevant in-context examples over detailed instruction. While semantic similarity gains were marginal, expected given the model’s baseline semantic capabilities, DFSL’s effect on syntactic structure suggests a promising direction for low-resource code generation tasks. This approach could benefit applications like automating simulations for digital twins, generating hardware description languages (e.g., Verilog), or translating legacy domain-specific scripts (e.g., LabVIEW or RTL). This research is limited in that we only test on Real-ESSI code, so a promising future direction can be extending this approach to other DSLs.

The challenge of generating DSL is not unique to the RES; DSLs are commonly used in modelling systems in various specialized fields. OpenSees, an open-sourced software modeling platform similar to the RES which simulates structural and geotechnical systems during earthquakes, uses a DSL (Peng, 2002). Modelica, a popular modeling language in engineering, enables the simulation of industrial systems through code rather than a GUI (Mattsson et al., 1998). In the life sciences, SBML and CellML are DSLs used to model complex biological and physiological processes (Hucka et al., 2003; Lloyd et al., 2008). These DSLs offer expert-level control, reproducibility, and computational efficiency. In addition, they enable users to take advantage of templates and frameworks, allowing them to replicate, share, and easily experiment with their work, allowing them to easily conduct parameter studies (OpenSees Project, 2013). However, the precision and control offered by DSL-usage comes at the expense of accessibility, as mentioned above, creating barriers and contributing to the steep learning curves that still limit the impact of simulators like the RES.

For example, the RES, compared to OpenSees, is much more specialized and sophisticated in the analysis of soil-structure interactions, using a more refined and domain-specific approach, offering higher levels of analysis and accuracy. This higher level of specialization is a reflection of its DSL usage. This contrasts with OpenSees’s usage of a more general-use scripting language (i.e. Tcl). Although the RES DSL requires more specialized syntax—specific to geomechanical and structural engineering—this allows for higher levels of manual control compared to the more general-purpose language and modular nature of OpenSees. However, OpenSees is much more widely used. This is due partly to its broader applicability across various structural engineering domains, but also because of its extensive community and the relative accessibility of its Tcl-based scripting environment—as opposed to DSL. This lowers the barrier-to-entry for new and unfamiliar users and thus enables easier integration and more common use.

As for the specialized retriever system for the RAG-enhanced LLM chatbot, our study demonstrates the iteration process and improvements necessary to increase retriever accuracy for large scale, technical literature such as Dr. Jeremić’s lecture notes on computational mechanics. Using the ChatGPT 4o model and other best practices sources, we were able to produce a proof-of-

concept specialized retriever using key methods such as hybrid chunking, similarity score modulation, metadata filtering, and the novel TOC-based section stitching. To our knowledge, this is the first retrieval system to implement TOC-based section stitching in a scientific context. Through our iterative trial and error process, we gained insights into best practices regarding chunking logic, TOC parsing, and system scalability and efficiency. We further introduced future research that could be applied in continuation of this study, such as the implementation of API based LLMs, hallucination and retriever accuracy evaluation metrics, cross-encoder reranking, other embedding models, and pseudo-relevance feedback. With more development, this foundation could lead to a fully-fledged RAG-enhanced AI assistant for the lecture notes and RES.

7 Acknowledgement

We would like to express sincere gratitude to Professor Boris Jeremić, for his invaluable guidance, advice, and mentorship throughout the duration of this project. We are also grateful for the Young Scholars Program at UC Davis for providing us with the opportunity and resources to engage in this research and exploration.

Additionally, we would like to acknowledge the use of AI tools such as OpenAI’s ChatGPT and Google’s Gemini, which were utilized to assist with literature review, code generation, troubleshooting, and the outlining of this paper. These tools were used as supportive aids under human supervision, with all final decisions, analysis, and validations performed manually. Again, we would like to convey our deepest thanks to Professor Boris Jeremic for taking his time to supervise us himself, providing us with insightful anecdotes, and for his flexibility. Lastly, we would like to thank our peers, family, and friends for their continued encouragement throughout this research experience.

8 References

- Béchar, P., & Ayala, O. M. (2024). Reducing hallucination in structured outputs via Retrieval-Augmented Generation. 228–238.
<https://doi.org/10.18653/v1/2024.naacl-industry.19>
- Bekele, Y. W. (2025). GeoSim.AI: AI assistants for numerical simulations in geomechanics (No. arXiv:2501.14186). arXiv.
<https://doi.org/10.48550/arXiv.2501.14186>
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D. (2020). Language Models are Few-Shot Learners (No. arXiv:2005.14165). arXiv.
<https://doi.org/10.48550/arXiv.2005.14165>

- Cassano, F., Gouwar, J., Lucchetti, F., Schlesinger, C., Freeman, A., Anderson, C. J., Feldman, M. Q., Greenberg, M., Jangda, A., & Guha, A. (2024). Knowledge Transfer from High-Resource to Low-Resource Programming Languages for Code LLMs (No. arXiv:2308.09895). arXiv. <https://doi.org/10.48550/arXiv.2308.09895>
- Charlotin, D. (n.d.). AI Hallucination Cases Database – Damien Charlotin. AI Hallucination Cases Database. Retrieved July 18, 2025, from <https://www.damiencharlotin.com/hallucinations/>
- Dahl, M., Magesh, V., Suzgun, M., & Ho, D. E. (2024). Large Legal Fictions: Profiling Legal Hallucinations in Large Language Models. *Journal of Legal Analysis*, 16(1), 64–93. <https://doi.org/10.1093/jla/laae003>
- Fan, L., Liu, F., & Chen, C. (2025). Domain adaptation of large language models for geotechnical applications (No. arXiv:2507.05613). arXiv. <https://doi.org/10.48550/arXiv.2507.05613>
- Ferrari, A., Abualhaija, S., & Arora, C. (2024). Model Generation with LLMs: From Requirements to UML Sequence Diagrams (No. arXiv:2404.06371). arXiv. <https://doi.org/10.48550/arXiv.2404.06371>
- Gao, T., Fisch, A., & Chen, D. (2021). Making Pre-trained Language Models Better Few-shot Learners. In C. Zong, F. Xia, W. Li, & R. Navigli (Eds.), *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)* (pp. 3816–3830). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2021.acl-long.295>
- Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., & Wang, H. (2024). Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Trans. Softw. Eng. Methodol.*, 33(8), 220:1–220:79. <https://doi.org/10.1145/3695988>
- Jeremić, B., & Yang, H. (2024). ENGINEERING ANALYSIS TOOLBOX, THE REAL-ESSI SIMULATOR SYSTEM.
- Joel, S., Wu, J. J., & Fard, F. H. (2024). A Survey on LLM-based Code Generation for Low-Resource and Domain-Specific Programming Languages (No. arXiv:2410.03981). arXiv. <https://doi.org/10.48550/arXiv.2410.03981>
- Kang, B., Kim, J., Yun, T.-R., & Kim, C.-E. (2024). Prompt-RAG: Pioneering Vector Embedding-Free Retrieval-Augmented Generation in Niche Domains, Exemplified by Korean Medicine (No. arXiv:2401.11246). arXiv. <https://doi.org/10.48550/arXiv.2401.11246>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474. https://proceedings.neurips.cc/paper_files/paper/2020/hash/

6b493230205f780e1bc26945df7481e5-Abstract.html

Li, B., & Gilbert, S. (2024). Artificial Intelligence awarded two Nobel Prizes for innovations that will shape the future of medicine. *Npj Digital Medicine*, 7(1), 336. <https://doi.org/10.1038/s41746-024-01345-9>

Li, H., Mourad, A., Zhuang, S., Koopman, B., & Zuccon, G. (2023). Pseudo Relevance Feedback with Deep Language Models and Dense Retrievers: Successes and Pitfalls. *ACM Transactions on Information Systems*, 41(3), 1–40. <https://doi.org/10.1145/3570724>

Li, Z., Wang, Z., Wang, W., Hung, K., Xie, H., & Wang, F. L. (2025). Retrieval-augmented generation for educational application: A systematic survey. *Computers and Education: Artificial Intelligence*, 8, 100417. <https://doi.org/10.1016/j.caeai.2025.100417>

Lin, S., Hilton, J., & Evans, O. (2022a). TruthfulQA: Measuring How Models Mimic Human Falsehoods. In S. Muresan, P. Nakov, & A. Villavicencio (Eds.), *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 3214–3252). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2022.acl-long.229>

Lin, S., Hilton, J., & Evans, O. (2022b). TruthfulQA: Measuring How Models Mimic Human Falsehoods (No. arXiv:2109.07958). arXiv. <https://doi.org/10.48550/arXiv.2109.07958>

Merola, C., & Singh, J. (2025). Reconstructing Context: Evaluating Advanced Chunking Strategies for Retrieval-Augmented Generation (No. arXiv:2504.19754). arXiv. <https://doi.org/10.48550/arXiv.2504.19754>

Mishra, P., Mahakali, A., & Venkataraman, P. S. (2024). SEARCHD - Advanced Retrieval with Text Generation using Large Language Models and Cross Encoding Re-ranking. *2024 IEEE 20th International Conference on Automation Science and Engineering (CASE)*, 975–980. <https://doi.org/10.1109/CASE59546.2024.10711642>

Nguyen, H.-T., Nguyen, T.-D., & Nguyen, V.-H. (2025, April 28). Enhancing Retrieval Augmented Generation with Hierarchical Text Segmentation Chunking — SpringerLink. https://link.springer.com/chapter/10.1007/978-981-96-4288-5_17

Orlanski, G., Xiao, K., Garcia, X., Hui, J., Howland, J., Malmaud, J., Austin, J., Singh, R., & Catasta, M. (2023). Measuring The Impact Of Programming Language Distribution (No. arXiv:2302.01973; Version 3). arXiv. <https://doi.org/10.48550/arXiv.2302.01973>

Pujar, S., Buratti, L., Guo, X., Dupuis, N., Lewis, B., Suneja, S., Sood, A., Nalawade, G., Jones, M., Morari, A., & Puri, R. (2023). Automated Code generation for Information Technology Tasks in YAML through Large Language Models (No. arXiv:2305.02783; Version 4). arXiv. <https://doi.org/10.48550/arXiv.2305.02783>

Sufi, F. (2024). Generative Pre-Trained Transformer (GPT) in Research: A Systematic Review on Data Augmentation. *Information*, 15(2), Article 2. <https://doi.org/10.3390/info15020099>

- Sun, K., Xu, Y. E., Zha, H., Liu, Y., & Dong, X. L. (2024). Head-to-Tail: How Knowledgeable are Large Language Models (LLMs)? A.K.A. Will LLMs Replace Knowledge Graphs? (No. arXiv:2308.10168). arXiv.
<https://doi.org/10.48550/arXiv.2308.10168>
- Wu, Z., Mitra, P., & Giles, C. L. (2013). Table of contents recognition and extraction for heterogeneous book documents. 2013 12th International Conference on Document Analysis and Recognition, 1205–1209.
<https://doi.org/10.1109/ICDAR.2013.244>
- Yenduri, G., M, R., G, C. S., Y, S., Srivastava, G., Maddikunta, P. K. R., G, D. R., Jhaveri, R. H., B, P., Wang, W., Vasilakos, A. V., & Gadekallu, T. R. (2023). Generative Pre-trained Transformer: A Comprehensive Review on Enabling Technologies, Potential Applications, Emerging Challenges, and Future Directions (No. arXiv:2305.10435). arXiv.
<https://doi.org/10.48550/arXiv.2305.10435>
- Zhang, X., Muralee, S., Cherupattamoolayil, S., & Machiry, A. (2024). On the effectiveness of Large Language Models for GitHub Workflows (No. arXiv:2403.12446; Version 1). arXiv.
<https://doi.org/10.48550/arXiv.2403.12446>